

3.2.1

1. 导入库

```
import onnxruntime as ort
import numpy as np
import scipy.special
from PIL import Image
```

- `onnxruntime`: 用于加载和运行ONNX模型
- `numpy`: 用于数值计算和数组操作
- `scipy.special`: 提供特殊数学函数, 这里使用softmax
- `PIL.Image`: Python图像处理库

2. 图像预处理函数

```
def preprocess_image(image, resize_size=256, crop_size=224, mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]):
```

定义预处理函数, 参数:

- `image`: 输入的PIL图像
- `resize_size=256`: 先调整到的尺寸
- `crop_size=224`: 最终裁剪的尺寸
- `mean`: 各通道的均值, 用于归一化
- `std`: 各通道的标准差, 用于归一化

```
image = image.resize((resize_size, resize_size), Image.BILINEAR)
```

将图像缩放到256×256像素, 使用双线性插值

```
w, h = image.size
left = (w - crop_size) / 2
top = (h - crop_size) / 2
image = image.crop((left, top, left + crop_size, top + crop_size))
```

计算中心裁剪位置并裁剪出224×224区域

```
image = np.array(image).astype(np.float32)
```

将图像转换为numpy数组并转为float32类型

```
image = image / 255.0
image = (image - mean) / std
```

将像素值归一化到[0,1]，然后用ImageNet的均值和标准差标准化

```
image = np.transpose(image, (2, 0, 1))  
image = image.reshape((1,) + image.shape)
```

将通道维度移到前面(C,H,W)，然后添加batch维度(1,C,H,W)

```
return image
```

返回处理后的图像张量

3. 加载模型

```
session = ort.InferenceSession('resnet.onnx')
```

创建ONNX运行时会话，加载resnet.onnx模型

4. 加载类别标签

```
labels_path = 'labels.txt'  
with open(labels_path) as f:  
    labels = [line.strip() for line in f.readlines()]
```

读取labels.txt文件，每行一个类别标签，存入列表

5. 获取输入输出名称

```
input_name = session.get_inputs()[0].name  
output_name = session.get_outputs()[0].name
```

获取模型输入和输出的名称，用于后续推理

6. 加载测试图像

```
image = Image.open('img_test.jpg').convert('RGB')
```

打开img_test.jpg并转换为RGB格式

7. 预处理图像

```
processed_image = preprocess_image(image)  
processed_image = processed_image.astype(np.float32)
```

调用预处理函数，并确保数据类型为float32

8. 模型推理

```
output = session.run([output_name], {input_name: processed_image})[0]
```

运行模型推理，输入处理后的图像，获取输出结果

9. 计算概率

```
probabilities = scipy.special.softmax(output, axis=-1)
```

对输出应用softmax函数，得到各类别的概率分布

10. 获取Top5结果

```
top5_idx = np.argsort(probabilities[0])[-5:][::-1]  
top5_prob = probabilities[0][top5_idx]
```

- 对概率排序，获取前5个最大值的索引(降序)
- 获取对应的概率值

11. 打印结果

```
print("Top 5 predicted classes:")  
for i in range(5):  
    print(f"{i+1}: {labels[top5_idx[i]]} - Probability: {top5_prob[i]}")
```

按概率从高到低打印前5个预测结果，包括类别名称和概率值

整体流程

1. 加载ONNX模型和类别标签
2. 加载并预处理图像(缩放、裁剪、归一化)
3. 运行模型推理
4. 处理输出结果(softmax、排序)
5. 显示最可能的5个预测类别

这段代码实现了一个完整的图像分类流程，使用ONNX格式的ResNet模型对输入图像进行分类预测。

3.2.2

1. 导入库

```
import onnxruntime
import numpy as np
from PIL import Image
```

- `onnxruntime`: 用于加载和运行ONNX模型
- `numpy`: 用于数值计算和数组操作
- `PIL.Image`: Python图像处理库

2. 加载ONNX模型

```
ort_session = onnxruntime.InferenceSession('mnist.onnx')
```

- 创建ONNX运行时会话
- 加载名为'mnist.onnx'的预训练模型文件
- 返回的 `ort_session` 对象将用于后续的推理

3. 加载测试图像

```
image = Image.open('img_test.png').convert('L')
```

- 打开名为'img_test.png'的图像文件
- `.convert('L')` 将图像转换为灰度图（单通道）
- MNIST数据集是黑白手写数字，所以需要灰度图

4. 图像预处理

```
image = image.resize((28, 28))
```

- 调整图像大小为28×28像素
- MNIST模型的输入要求就是这个尺寸

```
image_array = np.array(image, dtype=np.float32)
```

- 将PIL图像转换为numpy数组
- 指定数据类型为float32（模型要求的输入类型）

```
image_array = np.expand_dims(image_array, axis=0)
```

- 在第一个维度添加batch维度
- 形状从(28,28)变为(1,28,28)

- 因为模型预期输入是批量形式的

```
image_array = np.expand_dims(image_array, axis=0)
```

- 再次添加通道维度
- 形状从(1,28,28)变为(1,1,28,28)
- 虽然MNIST是单通道，但模型可能期望4D输入(batch,channel,height,width)

5. 准备模型输入

```
ort_inputs = {ort_session.get_inputs()[0].name: image_array}
```

- 获取模型输入节点的名称
- 创建输入字典: {输入节点名称: 处理后的图像数据}
- ONNX Runtime需要以这种格式接收输入

6. 执行预测

```
ort_outs = ort_session.run(None, ort_inputs)
```

- 运行模型推理
- 第一个参数 None 表示我们希望获取所有输出
- 第二个参数是输入数据字典
- 返回包含所有输出的列表

7. 获取预测结果

```
predicted_class = np.argmax(ort_outs[0])
```

- `ort_outs[0]` 获取第一个输出 (通常是分类logits)
- `np.argmax()` 找到概率最大的类别索引
- MNIST输出是0-9的数字概率，取最大值即为预测数字

8. 输出预测结果

```
print(f"Predicted class: {predicted_class}")
```

- 打印预测结果
- 输出格式示例: "Predicted class: 5"

代码整体流程

1. 加载预训练的MNIST ONNX模型
2. 加载并预处理测试图像：
 - 转换为灰度图
 - 调整大小为28×28
 - 转换为numpy数组并添加必要的维度
3. 准备模型输入数据
4. 运行模型推理
5. 解析输出结果（预测的数字类别）
6. 打印预测结果

注意事项

1. 这段代码专门用于MNIST手写数字分类
2. 输入图像需要是黑白数字，背景为黑色，数字为白色（与MNIST数据集一致）
3. 如果使用自定义图像，可能需要额外的预处理（如反色处理）
4. 模型输出是10个类别的概率分布（对应数字0-9）

3.2.3

1. 导入库

```
import numpy as np
from PIL import Image
import onnxruntime as ort
```

- `numpy`: 用于数值计算和数组操作
- `PIL.Image`: Python图像处理库，用于加载和处理图像
- `onnxruntime`: 用于加载和运行ONNX模型

2. 图像预处理函数

```
def preprocess(image_path):
```

定义预处理函数，参数：

- `image_path`: 图像文件路径

```
    input_shape = (1, 1, 64, 64)
```

定义模型期望的输入形状：

- 1: batch size (一次处理一张图像)
- 1: 通道数 (灰度图像)
- 64: 高度
- 64: 宽度

```
img = Image.open(image_path).convert('L')
```

- 打开指定路径的图像文件
- `.convert('L')` 将图像转换为灰度图 (单通道)

```
img = img.resize((64, 64), Image.ANTIALIAS)
```

- 调整图像大小为64×64像素
- `Image.ANTIALIAS`: 使用高质量的重采样滤波器

```
img_data = np.array(img, dtype=np.float32)
```

- 将PIL图像转换为numpy数组
- 指定数据类型为float32 (模型要求的输入类型)

```
img_data = np.expand_dims(img_data, axis=0) # 添加 batch 维度
img_data = np.expand_dims(img_data, axis=1) # 添加 channel 维度
```

- 第一次扩展添加batch维度 (从(64,64)变为(1,64,64))
- 第二次扩展添加通道维度 (从(1,64,64)变为(1,1,64,64))

```
assert img_data.shape == input_shape, f"Expected shape {input_shape}, but got {img_data.shape}"
```

- 验证最终形状是否与模型期望的输入形状一致
- 如果不一致，抛出错误信息

```
return img_data
```

返回处理后的图像数据，形状为(1,1,64,64)

3. 情感类别映射表

```
emotion_table = {'neutral':0, 'happiness':1, 'surprise':2, 'sadness':3, 'anger':4,
                 'disgust':5, 'fear':6, 'contempt':7}
```

创建情感标签与数字索引的映射字典：

- 键：情感名称 (字符串)
- 值：对应的类别索引 (整数)

4. 加载模型

```
ort_session = ort.InferenceSession('emotion-ferplus.onnx')
```

- 创建ONNX运行时会话
- 加载名为'emotion-ferplus.onnx'的情感识别模型

5. 加载并预处理图像

```
input_data = preprocess('img_test.png')
```

- 调用预处理函数处理'img_test.png'图像
- 返回符合模型输入要求的图像数据

6. 准备模型输入

```
ort_inputs = {ort_session.get_inputs()[0].name: input_data}
```

- 获取模型输入节点的名称
- 创建输入字典: {输入节点名称: 处理后的图像数据}

7. 运行模型推理

```
ort_outs = ort_session.run(None, ort_inputs)
```

- 执行模型推理
- `None`: 表示返回所有输出
- `ort_inputs`: 输入数据字典
- 返回包含模型输出的列表

8. 解析预测结果

```
predicted_label = np.argmax(ort_outs[0])
```

- `ort_outs[0]`: 获取第一个输出 (通常是分类概率)
- `np.argmax()`: 找到概率最大的类别索引

9. 获取情感名称

```
predicted_emotion = list(emotion_table.keys())[predicted_label]
```

- 将情感映射表的键 (情感名称) 转换为列表
- 使用预测的索引获取对应的情感名称

10. 输出预测结果

```
print(f"Predicted emotion: {predicted_emotion}")
```

打印预测的情感名称

代码整体流程

1. 定义图像预处理函数，将图像转换为模型所需的格式
2. 创建情感类别与数字标签的映射表
3. 加载ONNX格式的情感识别模型
4. 加载并预处理测试图像
5. 准备模型输入数据
6. 运行模型推理
7. 解析输出结果，找到概率最高的情感类别
8. 将数字标签转换为情感名称
9. 打印预测结果

关键点说明

1. 此代码专门用于FER+情感识别模型，该模型识别8种基本情感
2. 输入图像必须是64×64像素的灰度图
3. 模型输出是一个8维向量，表示每种情感的概率
4. 使用 `np.argmax()` 获取概率最高的情感索引
5. 通过映射表将索引转换为情感名称

3.2.4

下面我将逐行详细解释这段代码的功能、参数含义以及各个部分的作用：

1. 导入库

```
import onnxruntime as ort
import numpy as np
import scipy.special
from PIL import Image
```

- `onnxruntime`: 用于加载和运行ONNX模型
- `numpy`: 用于数值计算和数组操作
- `scipy.special`: 提供特殊数学函数，这里使用softmax

- `PIL.Image`: Python图像处理库

2. 图像预处理函数

```
def preprocess_image(image, resize_size=256, crop_size=224, mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]):
```

定义预处理函数，参数：

- `image`: 输入的PIL图像对象
- `resize_size=256`: 先调整到的尺寸（默认256×256）
- `crop_size=224`: 中心裁剪的尺寸（默认224×224）
- `mean`: 各通道的均值（ImageNet数据集）
- `std`: 各通道的标准差（ImageNet数据集）

```
image = image.resize((resize_size, resize_size), Image.BILINEAR)
```

将图像缩放到指定大小，使用双线性插值

```
w, h = image.size
left = (w - crop_size) / 2
top = (h - crop_size) / 2
image = image.crop((left, top, left + crop_size, top + crop_size))
```

计算中心位置并裁剪出指定大小的区域

```
image = np.array(image).astype(np.float32)
```

将PIL图像转换为numpy数组并转为float32类型

```
image = image / 255.0
```

将像素值从[0,255]归一化到[0,1]

```
image = (image - mean) / std
```

使用ImageNet的均值和标准差进行标准化

```
image = np.transpose(image, (2, 0, 1))
```

将图像数据从(H,W,C)转置为(C,H,W)格式

```
image = image.reshape((1,) + image.shape)
```

添加batch维度，形状变为(1,C,H,W)

```
return image
```

返回处理后的4维张量

3. 加载模型

```
session = ort.InferenceSession('flower-detection.onnx')
```

创建ONNX运行时会话，加载花朵检测模型

4. 加载类别标签

```
with open('labels.txt') as f:  
    labels = [line.strip() for line in f.readlines()]
```

读取标签文件，每行一个类别名称，去除首尾空格

5. 获取输入输出名称

```
input_name = session.get_inputs()[0].name  
output_name = session.get_outputs()[0].name
```

获取模型输入和输出节点的名称

6. 加载测试图像

```
image = Image.open('flower_test.png').convert('RGB')
```

打开测试图像并确保是RGB格式

7. 预处理图像

```
processed_image = preprocess_image(image)  
processed_image = processed_image.astype(np.float32)
```

调用预处理函数，并确保数据类型为float32

8. 模型推理

```
output = session.run([output_name], {input_name: processed_image})[0]
```

运行模型，获取输出结果

9. 计算类别概率

```
accuracy = scipy.special.softmax(output, axis=-1)
```

对模型输出应用softmax，得到各类别概率分布

10. 解析预测结果

```
predicted_idx = np.argmax(accuracy[0])
```

找到概率最大的类别索引

```
prob_percentage = accuracy[0, predicted_idx] * 100
```

将最高概率转换为百分比形式

```
predicted_label = labels[predicted_idx]
```

根据索引获取对应的类别名称

11. 输出结果

```
print(f"Predicted class: {predicted_label}, Accuracy: {prob_percentage:.2f}%")
```

格式化输出预测结果，保留2位小数

代码整体流程

1. 加载ONNX模型和类别标签
2. 加载并预处理图像：
 - 调整大小 → 中心裁剪 → 归一化 → 标准化 → 维度转换
3. 运行模型推理
4. 处理输出结果：
 - 计算softmax概率 → 获取最高概率类别 → 转换为百分比
5. 输出预测结果

关键点说明

1. 预处理流程符合ImageNet标准
2. 模型输入要求：batch×3×224×224的float32张量
3. 输出是各类别的logits，需通过softmax转换为概率
4. 最终输出包含类别名称和置信度百分比

3.2.5

1. 导入库

```
import os
import time
import cv2
import numpy as np
import vision.utils.box_utils_numpy as box_utils
import onnxruntime as ort
```

- `os`: 用于文件和目录操作
- `time`: 用于时间相关操作，这里用来计算推理时间
- `cv2`: OpenCV库，用于图像处理
- `numpy`: 数值计算库
- `box_utils`: 包含边界框处理工具的自定义模块
- `onnxruntime`: ONNX模型推理运行时

2. 预测函数 `predict` 函数详细解释

这个函数用于对目标检测模型的原始输出进行后处理，主要完成以下任务：

函数参数说明

参数	类型	说明
<code>width</code>	<code>int</code>	原始图像的宽度（像素）
<code>height</code>	<code>int</code>	原始图像的高度（像素）
<code>confidences</code>	<code>np.array</code>	模型输出的置信度数组（每个边界框的类别概率）
<code>boxes</code>	<code>np.array</code>	模型输出的边界框数组（归一化坐标）
<code>prob_threshold</code>	<code>float</code>	置信度阈值，过滤低置信度预测
<code>iou_threshold</code>	<code>float</code>	IoU 阈值，用于非极大值抑制（NMS）
<code>top_k</code>	<code>int</code>	每个类别保留的最大检测框数量

函数执行步骤详解

1. 提取单个图像的结果

```
boxes = boxes[0]
confidences = confidences[0]
```

模型输出通常是批量处理结果，这里只处理第一个图像的结果（假设批大小为1）

2. 初始化结果容器

```
picked_box_probs = [] # 存储筛选后的边界框和置信度
picked_labels = [] # 存储对应的类别标签
```

3. 遍历每个类别（跳过背景类0）

```
for class_index in range(1, confidences.shape[1]):
```

4. 筛选置信度高于阈值的预测

```
probs = confidences[:, class_index] # 获取当前类别的所有置信度
mask = probs > prob_threshold # 创建置信度阈值掩码
probs = probs[mask] # 应用掩码筛选置信度
```

5. 跳过没有有效预测的类别

```
if probs.shape[0] == 0:
    continue
```

6. 获取符合条件的边界框

```
subset_boxes = boxes[mask, :] # 应用相同的掩码筛选边界框
```

7. 组合边界框和置信度

```
box_probs = np.concatenate([subset_boxes, probs.reshape(-1, 1)], axis=1)
```

创建形状为 `[N, 5]` 的数组，每行包含 `[x_min, y_min, x_max, y_max, confidence]`

8. 应用非极大值抑制（NMS）

```
box_probs = box_utils.hard_nms(box_probs,
                                iou_threshold=iou_threshold,
                                top_k=top_k)
```

去除重叠度高的冗余边界框，保留最具代表性的检测结果

9. 保存筛选结果

```
picked_box_probs.append(box_probs)
picked_labels.extend([class_index] * box_probs.shape[0])
```

10. 处理无检测结果的情况

```
if not picked_box_probs:
    return np.array([]), np.array([]), np.array([])
```

11. 合并所有类别的结果

```
picked_box_probs = np.concatenate(picked_box_probs)
```

12. 将归一化坐标转换为原始图像尺寸

```
picked_box_probs[:, 0] *= width # x_min  
picked_box_probs[:, 1] *= height # y_min  
picked_box_probs[:, 2] *= width # x_max  
picked_box_probs[:, 3] *= height # y_max
```

13. 返回最终结果

```
return (  
    picked_box_probs[:, :4].astype(np.int32), # 边界框坐标 (整数像素值)  
    np.array(picked_labels), # 类别标签  
    picked_box_probs[:, 4] # 置信度  
)
```

函数输出说明

该函数返回三个数组组成的元组：

1. **边界框坐标**：形状为 `[N, 4]` 的整数数组，每行表示一个边界框的 `[x_min, y_min, x_max, y_max]`
2. **类别标签**：形状为 `[N]` 的整数数组，每个元素表示对应边界框的类别索引
3. **置信度**：形状为 `[N]` 的浮点数数组，每个元素表示对应预测的置信度

这个函数是目标检测流程中的关键后处理步骤，负责将模型的原始输出转换为可直接使用的检测结果。

3. 类别名称读取

```
class_names = [name.strip() for name in open('voc-model-labels.txt').readlines()]
```

- 从'voc-model-labels.txt'文件中读取类别名称列表，并去除每行首尾的空白字符

4. ONNX模型加载

```
ort_session = ort.InferenceSession('version-RFB-320.onnx')
```

- 创建ONNX Runtime推理会话，加载'version-RFB-320.onnx'模型

5. 获取输入名称

```
input_name = ort_session.get_inputs()[0].name
```

- 获取模型输入节点的名称

6. 路径和参数设置

```
result_path = "./detect_imgs_results_onnx"  
threshold = 0.7  
path = "imgs"  
sum = 0
```

- `result_path`: 结果保存路径
- `threshold`: 置信度阈值(0.7)
- `path`: 待检测图像目录
- `sum`: 用于统计总检测框数

7. 创建结果目录

```
if not os.path.exists(result_path):  
    os.makedirs(result_path)
```

- 如果结果目录不存在则创建

8. 处理每张图像

```
for file_path in.listdir:  
    img_path = os.path.join(path, file_path)  
    orig_image = cv2.imread(img_path)
```

- 遍历目录中的每个文件
- 拼接完整图像路径
- 用OpenCV读取图像

9. 图像预处理

```
image = cv2.cvtColor(orig_image, cv2.COLOR_BGR2RGB)  
image = cv2.resize(image, (320, 240))  
image_mean = np.array([127, 127, 127])  
image = (image - image_mean) / 128  
image = np.transpose(image, [2, 0, 1])  
image = np.expand_dims(image, axis=0)  
image = image.astype(np.float32)
```

- BGR转RGB
- 调整大小为320x240
- 定义均值并归一化
- 转换维度顺序为(C,H,W)
- 增加batch维度
- 转换为float32类型

10. 模型推理

```
time_time = time.time()
confidences, boxes = ort_session.run(None, {input_name: image})
print("cost time:{}".format(time.time() - time_time))
```

- 记录开始时间
- 运行模型推理
- 打印推理耗时

11. 后处理

```
boxes, labels, probs = predict(orig_image.shape[1], orig_image.shape[0], confidences, boxes, threshold)
```

- 调用predict函数处理模型输出

12. 绘制结果

```
for i in range(boxes.shape[0]):
    box = boxes[i, :]
    label = f"{class_names[labels[i]]}: {probs[i]:.2f}"
    cv2.rectangle(orig_image, (box[0], box[1]), (box[2], box[3]), (255, 255, 0), 4)
    cv2.imwrite(os.path.join(result_path, file_path), orig_image)
```

- 遍历每个检测框
- 获取框坐标
- 生成标签文本(类别+置信度)
- 在图像上绘制矩形框
- 保存结果图像

13. 统计总数

```
sum += boxes.shape[0]
print("sum:{}".format(sum))
```

- 累加检测框数量
- 打印总数

主要函数总结：

1. `predict()` : 处理模型输出，包括：
 - 过滤低置信度检测
 - 执行非极大值抑制(NMS)
 - 将归一化坐标转换为原始图像坐标

2. 预处理流程：

- 颜色空间转换
- 尺寸调整
- 归一化
- 维度转换

3. 后处理流程：

- 运行模型推理
- 处理输出
- 绘制结果
- 保存图像